

Programación funcional

Lambdas, métodos de referencia y streams

4.1 ¿Qué es la programación funcional?

Si le pides a un compañero que te avise cuando lleguen pedidos de más de 50€, no le explicas paso a paso cómo mirar la lista: le das una regla ("que supere los 50€") y él la aplica.

Eso es lo que vas a hacer aquí: en vez de escribir tú el bucle, le das a Java la regla.

Es un paradigma que se apoya en dos ideas: los datos no cambian de valor una vez creados (inmutabilidad) y las funciones se tratan como datos, encadenando la salida de una como entrada de la siguiente.

Cuando filtras o transformas una lista con estas herramientas, la lista original no se toca: se genera un resultado nuevo, y personas sigue teniendo los mismos elementos que antes.

4.1 Imperativo vs funcional

Compara las dos formas de resolver el mismo problema: dada una lista de personas, obtener solo las mayores de edad.

Imperativo

```
List<Persona> adultos =  
    new ArrayList<>();  
for (int i = 0;  
    i < personas.size(); i++) {  
    if (personas.get(i)  
        .getEdad() >= 18)  
        adultos.add(  
            personas.get(i));  
}
```

Funcional

```
List<Persona> adultos =  
    personas.stream()  
        .filter(p ->  
            p.getEdad() >= 18)  
        .toList();
```

La versión funcional es más compacta y encadena operaciones, a costa de ser más propensa a errores si no tienes claro qué hace cada pieza.

4.2 Funciones lambda — sintaxis

Una función lambda (o anónima) es una función sin nombre y sin clase propia, pensada para operaciones simples que se usan una sola vez.

```
(int a, int b) -> a + b
```

Los parámetros van antes de la flecha -> (sin indicar el tipo, Java lo deduce solo) y el código de la función va después. Nos ahorra crear una clase entera solo para pasar un trozo de comportamiento.

(int a, int b)	->	a + b
los parámetros de entrada	separa parámetros y cuerpo	el cuerpo: lo que devuelve

4.2 Cuidado: una lambda no tiene tipo fijo

Java necesita saber qué interfaz funcional estás implementando (cuántos parámetros, de qué tipo, qué debe devolver) para darle sentido a la lambda. Esa información la saca del contexto: el tipo de la variable, o el tipo que espera el método al que se la pasas.

```
Comparator<Persona> comp = (p1, p2) -> p2.getEdad() - p1.getEdad();
```

Por eso una lambda nunca aparece "sola": siempre va asignada a algo o pasada como argumento a algo.

4.2 Ejemplo: ordenar personas por edad

Antes de las lambdas, ordenar con un criterio propio exigía implementar `Comparator` en una clase aparte:

Antes de las lambdas

```
class ComparadorPersona implements
    Comparator<Persona> {
    @Override
    public int compare(
        Persona p1, Persona p2) {
        return p2.getEdad()
            - p1.getEdad();
    }
}

personas.sort(
    new ComparadorPersona());
```

Con una lambda

```
personas.sort((p1, p2) ->
    p2.getEdad() - p1.getEdad());
```

4.3 Métodos de referencia

Un método de referencia es una forma abreviada de escribir una lambda que se limita a llamar a un método que ya existe, con la sintaxis `Clase::metodo`.

Tipo	Ejemplo	Equivale a
Método estático	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>
Método de instancia	<code>Persona::getNombre</code>	<code>p -> p.getNombre()</code>
Método de un objeto arbitrario	<code>Integer::sum</code>	<code>(a, b) -> a + b</code>

4.3 Ejemplo de métodos de referencia

Los tres tipos de la tabla, usados dentro de un stream real:

```
List<String> nombres = List.of("ana", "Luis", "MARÍA");

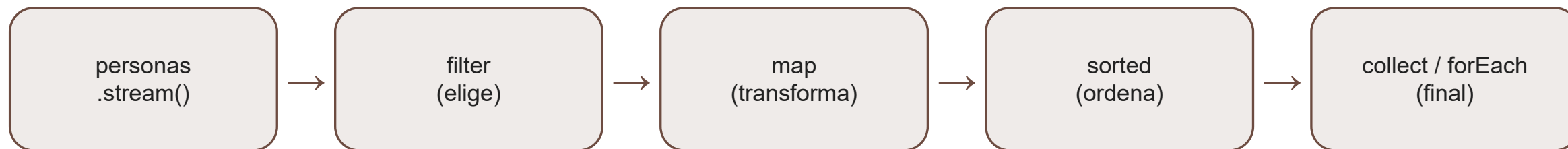
// Estático: String::toUpperCase
nombres.stream().map(String::toUpperCase)
        .forEach(System.out::println); // ANA, LUIS, MARÍA

// De instancia: Persona::getNombre
List<String> soloNombres = personas.stream()
        .map(Persona::getNombre).toList();

// De un objeto arbitrario: Integer::sum
int total = List.of(10, 20, 30).stream()
        .reduce(0, Integer::sum); // 60
```


4.4 Streams — la cadena de montaje

Un stream es un envoltorio sobre una colección que encadena operaciones sin modificar la colección original. Piensa en él como una cadena de montaje: personas entra por un lado, cada estación le hace algo, y al final sale un resultado.



Tipo	Qué hace	Ejemplos
Intermedia	recibe un stream y devuelve otro, para seguir encadenando	filter, map, sorted
Final	cierra el stream y devuelve un resultado que ya no es un stream	collect, forEach, reduce

En cuanto llamas a una operación final (collect, forEach, reduce...) el stream se cierra. Si intentas reutilizar esa misma variable, Java lanza `IllegalStateException`. Si necesitas recorrer los datos de dos formas, vuelve a llamar a `.stream()` sobre la colección original.

4.4 filter — filtrar por un criterio

Se queda solo con los elementos que cumplen la condición indicada, igual que haría un if dentro de un bucle, pero sin bucle.

Imperativo

```
List<Persona> adultos =  
    new ArrayList<>();  
for (Persona p : personas) {  
    if (p.getEdad() >= 18) {  
        adultos.add(p);  
    }  
}
```

Funcional

```
Stream<Persona> adultos =  
    personas.stream()  
        .filter(p ->  
            p.getEdad() >= 18);
```

4.4 map — transformar cada elemento

Aplica una función a cada elemento y devuelve un stream con los resultados, no con las personas originales.

Imperativo

```
List<Integer> edades =  
    new ArrayList<>();  
for (Persona p : personas) {  
    if (p.getEdad() >= 18) {  
        edades.add(p.getEdad());  
    }  
}
```

Funcional

```
Stream<Integer> edades =  
    personas.stream()  
        .filter(p ->  
            p.getEdad() >= 18)  
        .map(Persona::getEdad);
```

4.4 sorted — ordenar los elementos

Ordena el stream según el Comparator que le pases, igual que hacía sort en el apartado anterior.

Imperativo

```
List<Persona> adultos =  
    new ArrayList<>();  
for (Persona p : personas) {  
    if (p.getEdad() >= 18)  
        adultos.add(p);  
}  
Collections.sort(adultos,  
    (p1, p2) ->  
        p2.getEdad() - p1.getEdad());
```

Funcional

```
Stream<Persona> ordenadas =  
    personas.stream()  
        .filter(p ->  
            p.getEdad() >= 18)  
        .sorted((p1, p2) ->  
            p2.getEdad() - p1.getEdad());
```

4.4 collect y forEach — operaciones finales

toList() → convierte el resultado en una lista

```
List<Integer> edades = personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .map(Persona::getEdad)
    .toList(); // atajo de .collect(Collectors.toList())
```

collect() → une el resultado en un texto

```
String texto = personas.stream()
    .map(Persona::getEdad)
    .collect(Collectors.joining(", ", "Adultos: ", ""));
```

forEach() → recorre el resultado para hacer algo con él

```
personas.stream()
    .filter(p -> p.getEdad() >= 18)
    .forEach(System.out::println);
```

4.4 reduce — reducir a un solo valor

Combina todos los elementos en uno solo usando una función de acumulación. Es el equivalente funcional de un acumulador que actualizas dentro de un bucle:

Imperativo

```
List<Integer> edades =  
    Arrays.asList(20, 30, 40);  
  
int suma = 0;  
for (int edad : edades) {  
    suma += edad;  
}
```

Funcional

```
List<Integer> edades =  
    Arrays.asList(20, 30, 40);  
  
int suma = edades.stream()  
    .reduce(0, Integer::sum); // 90
```

El valor inicial debe ser el que no cambia el resultado al combinarlo: 0 para sumar, 1 si multiplicaras. Además sirve de resultado por defecto si el stream estuviera vacío. Por eso existe una versión sin valor inicial, que devuelve un Optional en vez de un int.

4.4 average — la media de un stream

```
double mediaEdadAdultos = personas.stream()  
    .filter(p -> p.getEdad() >= 18)  
    .mapToInt(Persona::getEdad)  
    .average()  
    .getAsDouble();
```

`average()` no devuelve un `double`, sino un `OptionalDouble`: si el stream estuviera vacío no habría media que calcular, y `getAsDouble()` lanzaría una excepción.

No hace falta memorizar todas las operaciones de golpe: con `filter`, `map`, `sorted`, `collect` y `forEach` resuelves la mayoría de los casos.

Cómo encajan todas las piezas

- Una lambda es una función sin nombre que escribes donde antes hacía falta una clase entera.
- Una interfaz funcional (Comparator, Predicate, Function...) es el "molde" que le dice a Java qué forma debe tener esa lambda.
- Un método de referencia es un atajo para cuando la lambda solo llama a un método que ya existe.
- Un stream encadena lambdas, interfaces funcionales y métodos de referencia para procesar una colección sin bucles.

Si te pierdes con una lambda concreta, vuelve a esta lista: casi siempre el bloqueo está en no tener claro cuál de estas cuatro piezas estás mirando.